Where In-Context Grammar Learning Appears in Mistral-7B

Dor · October 20, 2026

Summary

The paper shows that a frozen LLM (Mistral-7B) learns an artificial grammar from examples in its prompt alone, with no weight updates. I reproduced that result, then asked a new question the paper does not address: **at which of the model's 32 layers does this learning become visible?**

- **Reproduction:** the main results match the paper. On AGL the model reaches about 85% of the theoretical bound (0.39 of 0.46; paper: 79%); on SRTT it is near perfect (paper: 92%). All controls behave as reported.
- **Main new finding:** for both grammars, the learned structure appears **abruptly at layer 19**. Layers 0–18 are at or near chance; within one layer, AGL jumps from 2% to 40% of the bound and SRTT from 8% to 53%. A tuned lens, which reads inner layers more faithfully, shows the same jump.
- **After layer 19,** performance keeps improving with depth up to the last layer.
- **The model "knows" before it can "say" it:** the correct answer is preferred among the candidates from layer 19, but only becomes the model's actual word output from layer ~22.

Setup

The model reads one long stream of words built from 4 random English words plus `END`. At every position we record the probability it gave to the word that actually came next. The stream has three parts:

1. **Random** (~420 words): no rules, so the best possible guess is 1 in 5 = 0.2.
2. **Grammar** (~2,450 words for AGL, 2,880 for SRTT): the same words now follow rules, so there is something to learn.
3. **Random again** (~410 words): a control. If the model learned the rules, its advantage disappears here.

Two grammars, both classic human statistical-learning tasks:

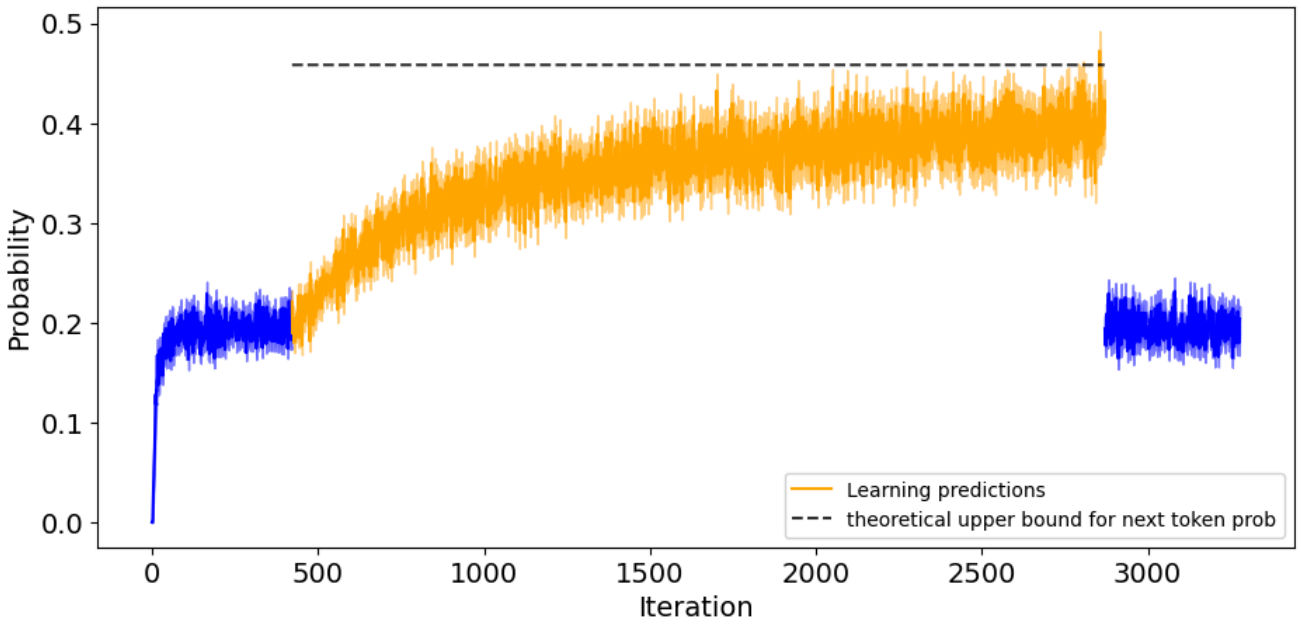
- **SRTT** (Serial Reaction Time Task): one fixed 8-word sequence plus END, repeated, like a song on repeat. Fully predictable, so the ideal score is 1.
- **AGL** (Artificial Grammar Learning): a 5-state rule system where each next word is chosen at random from 2–3 allowed options. The rules are learnable but the choice is not, so the ideal score is about 0.46.

Every result is the average of 100 runs, each with a different random vocabulary, so nothing depends on particular words.

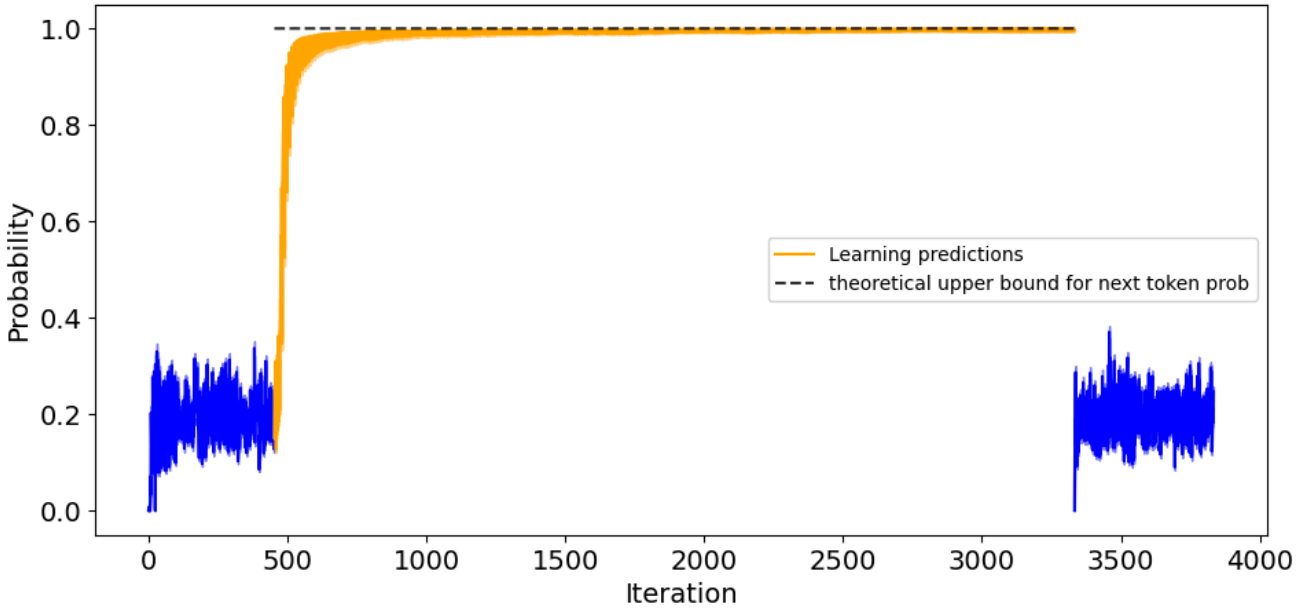
Part 1: Reproducing the paper

These plots measure only the model's final output, as in the paper.

Learning curves

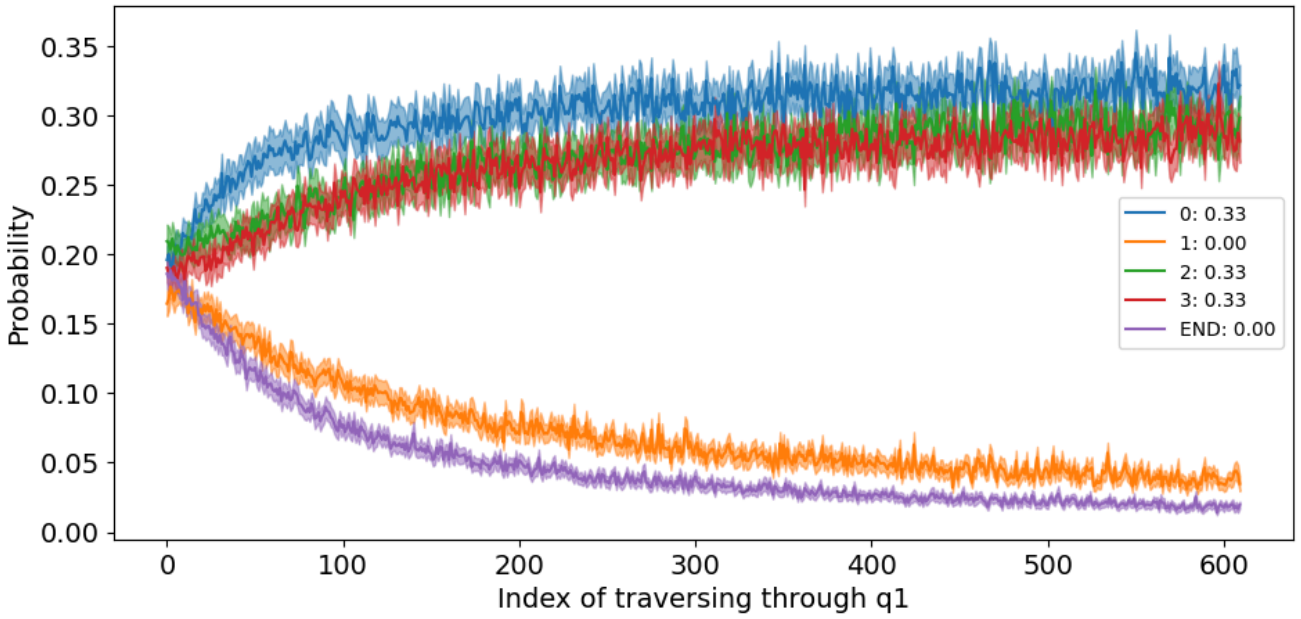


AGL. At chance (0.2) in the random part, climbs to ~0.39 during the grammar (bound 0.46, dashed), and drops straight back to 0.2 when the rules disappear.



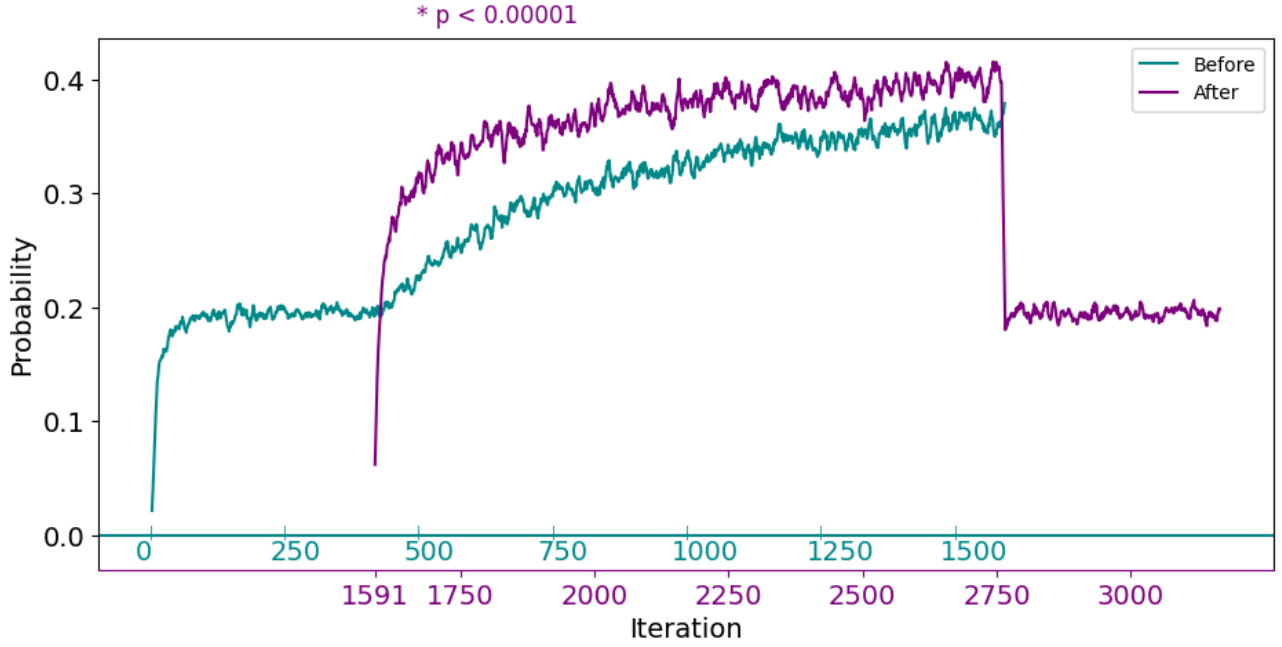
SRTT. Reaches ~0.95 within about 10 repetitions and ~0.99 by the end; back to 0.2 afterwards.

The model learns the full rule structure

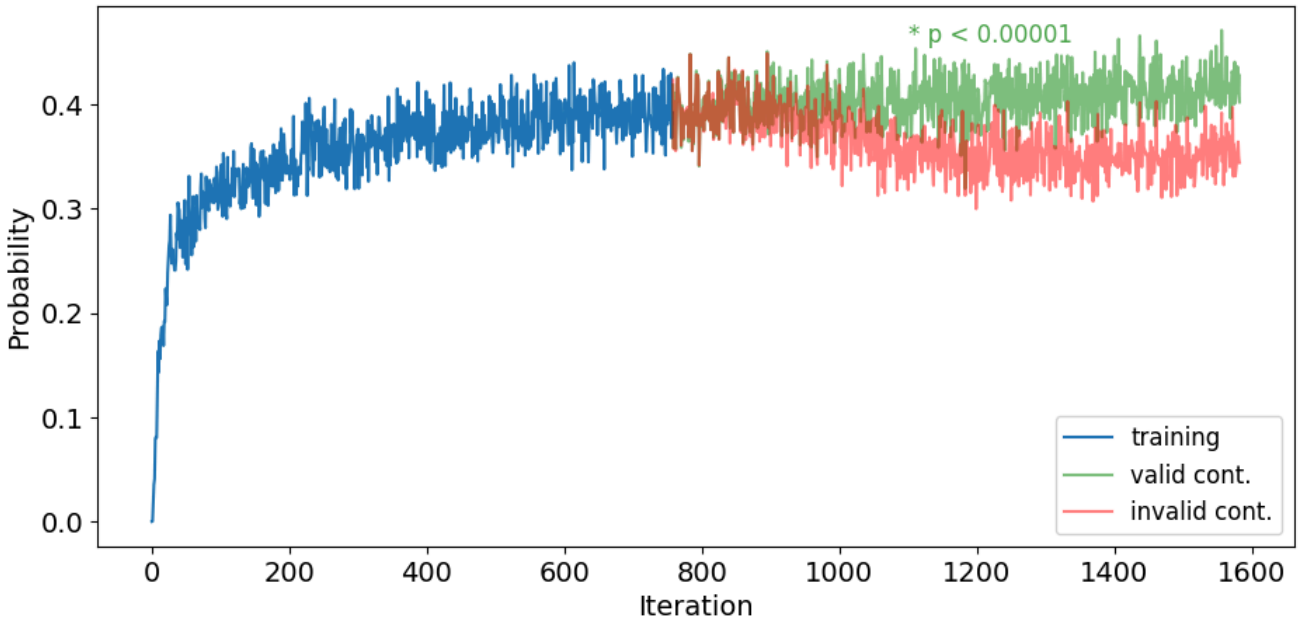


At one AGL state (q1), three words are allowed (true probability 0.33 each) and two are forbidden. The allowed ones rise toward 0.3; the forbidden ones fall toward 0.02–0.04. The model learns which options are allowed, not a single association.

It learns the rules, not the words



Vocabulary switch. Halfway through, the same rules continue with 4 new words. The second time (purple), the model reaches ~0.3 within ~100 words; the first time (teal) it needed ~800. It reuses the abstract structure.

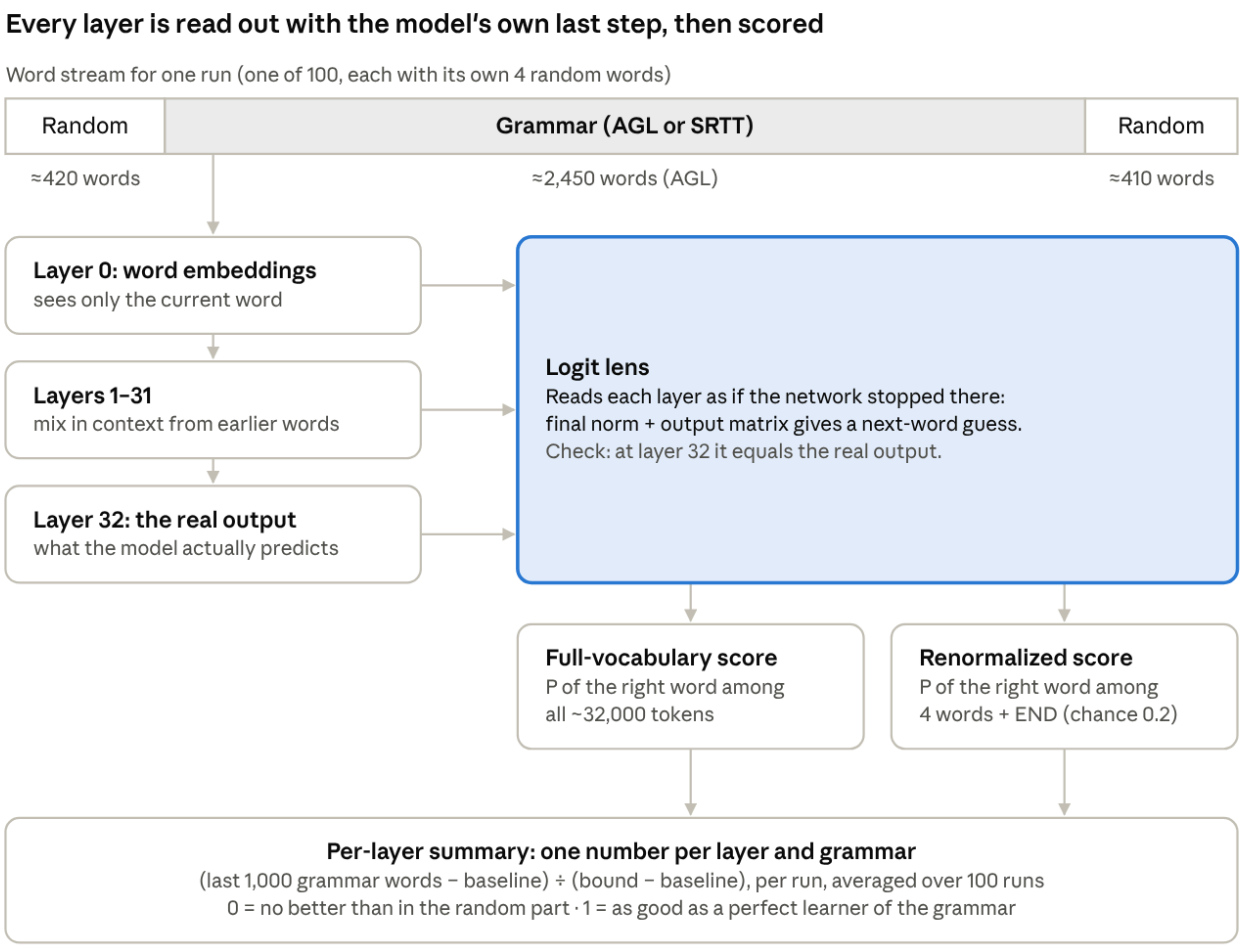


Valid vs. broken sequences. After reading valid sequences, the model scores new, unseen valid sequences at ~0.41 and the same sequences with two neighboring words swapped at ~0.35. It generalizes the rules and notices violations.

Part 2: How the per-layer measurement works

The question. The paper only looks at the model's final answer. Inside, each word passes through 32 layers, each refining the previous one. We ask: *if the model had to answer at layer X, how good would its guess be?*

The method (logit lens). The model's last step converts its internal state into a guess over words (a normalization followed by the output matrix). We apply that same last step to the internal state after **each** layer, as if the network stopped there. Nothing is trained or changed.



Same test as Part 1, at every layer. The same 100 word streams, one forward pass each; at every position and every layer we record the probability of the correct next word. At layer 32 this reproduces the Part 1 curves (to within 0.002).

Two scores per layer:

- **Full-vocabulary:** probability of the correct word out of all ~32,000 tokens. This is what the model would actually say.
- **Renormalized:** among only the 5 possible answers (4 words + END), the share that goes to the correct one. Chance = 0.2. Early layers spread their probability over unrelated tokens, so this asks whether a layer already *prefers* the right answer even before it can output it as a word.

One summary number per layer: (phase mean − baseline) ÷ (bound − baseline), i.e. how much of the possible improvement the layer achieved.

- **Phase mean:** the layer's average probability for the correct next word over the last 1,000 grammar words, when learning is mostly done.
- **Baseline:** the same average over the first random part, where nothing can be learned.
- **Bound:** the best a perfect learner could score (0.46 for AGL, 1.0 for SRTT).

0 = no better than in the random part; 1 = reached the bound. Computed per run, then averaged over the 100 runs. Each layer is compared with its own baseline, so a layer that

merely likes the 4 words gets no credit, and layers with different starting levels are compared fairly.

Example (AGL, renormalized)	Baseline	Phase mean	Bound	Calculation	Score
Layer 19	0.204	0.305	0.458	0.101 ÷ 0.254	0.40
Layer 32	0.206	0.405	0.458	0.199 ÷ 0.252	0.79

Checks that passed:

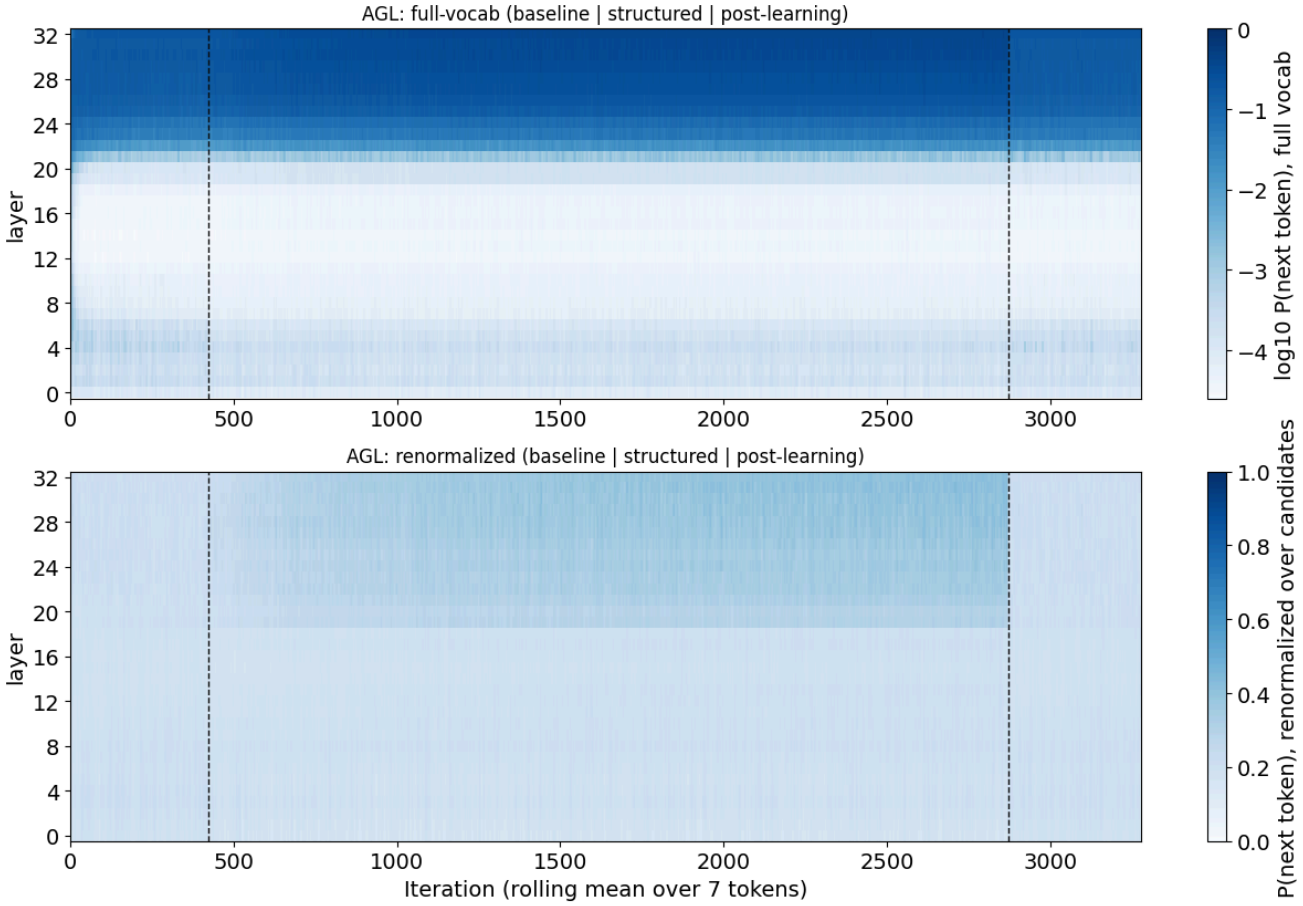
- Layer 32 through the lens reproduces the model's real output exactly (difference 0).
- Layer 0 sees only the current word, so it cannot learn from context: its score does not drift during the grammar part (all four checks OK).
- In the random part, the renormalized score is ~0.19–0.22 at every layer, as expected for chance.

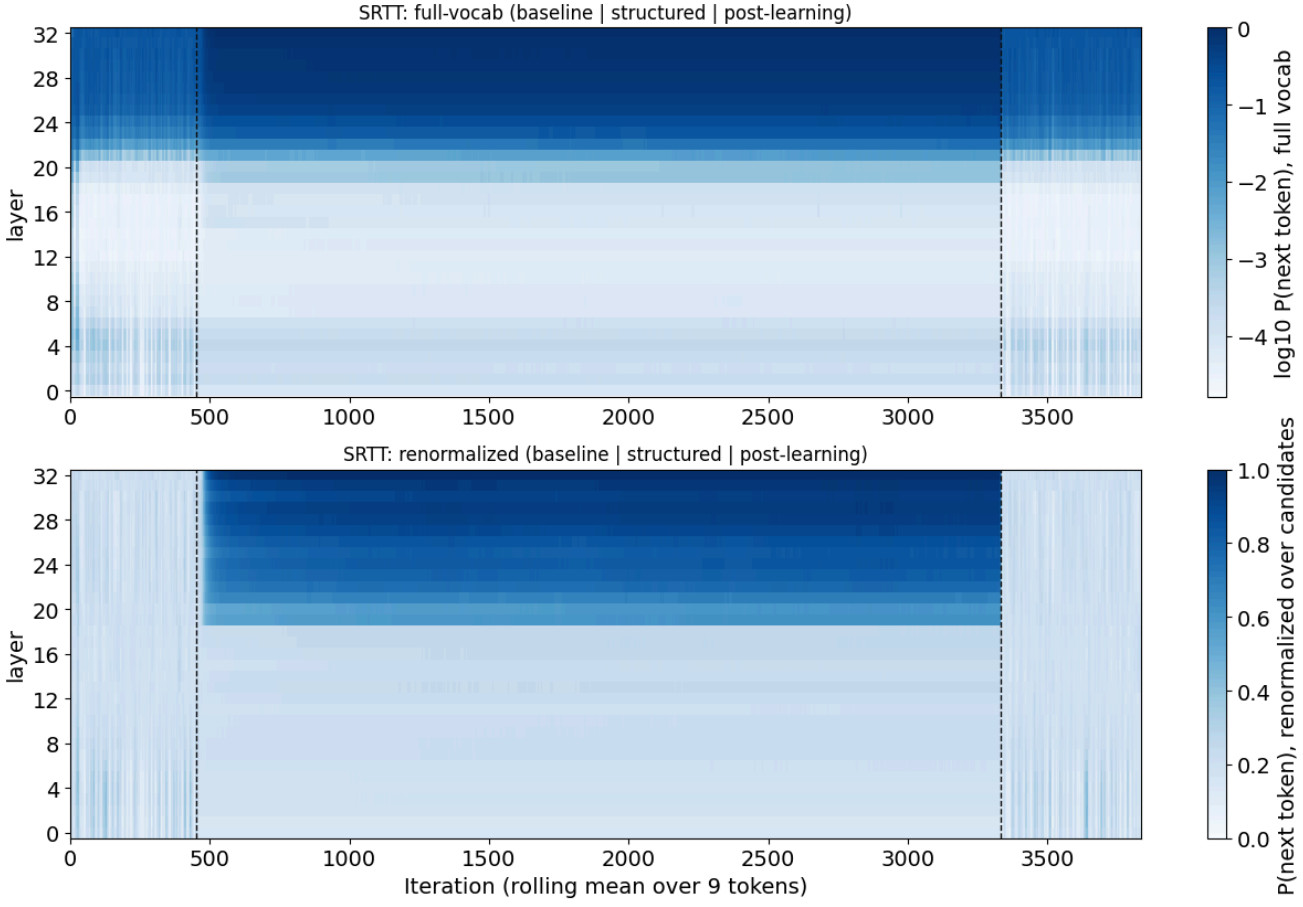
Caveat. The lens uses a last step trained for the last layer only, so it may fail to read information that earlier layers hold in another form. The claim is "readable at layer X", not "computed at layer X".

Part 2: Results

Heatmaps: where and when the grammar is visible

Each row is a layer (0 bottom, 32 top), each column a position in the word stream; dashed lines separate random | grammar | random. Darker = higher probability of the correct next word. Top panel full-vocabulary (log scale), bottom panel renormalized.

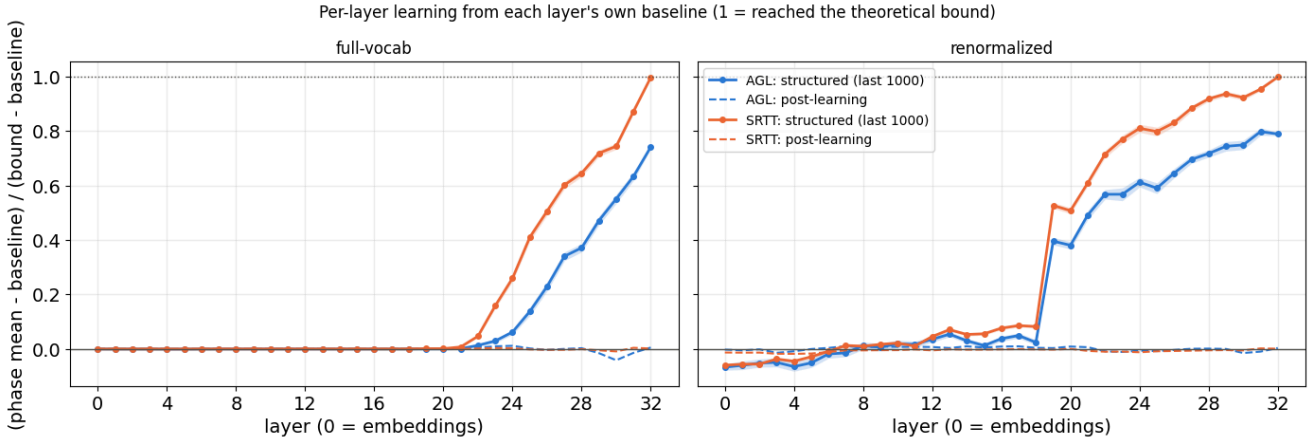




- **Layers 0–18** look the same in all three parts: at chance, no learning.

- **From layer 19 up,** the grammar part turns darker (and keeps darkening as more examples are read), then fades back to chance when the rules stop.
- **Full-vocabulary** shows learning only from layer ~22: the correct word becomes the model's actual output a few layers after it is already preferred.

Summary per layer



Solid lines: grammar part (0 = no learning, 1 = perfect learner). Dashed lines: the random part after the grammar (control).

Layer	AGL, renormalized	SRTT, renormalized	AGL, full-vocab	SRTT, full-vocab
0–11	−0.06 to 0.02	−0.06 to 0.02	0	0
12–18	≈ 0.02–0.05	0.05–0.09	0	0
19	**≈ 0.40**	**0.53**	0	0
24	≈ 0.61	0.81	0.06	0.26
32	≈ 0.79	1.00	0.74	1.00

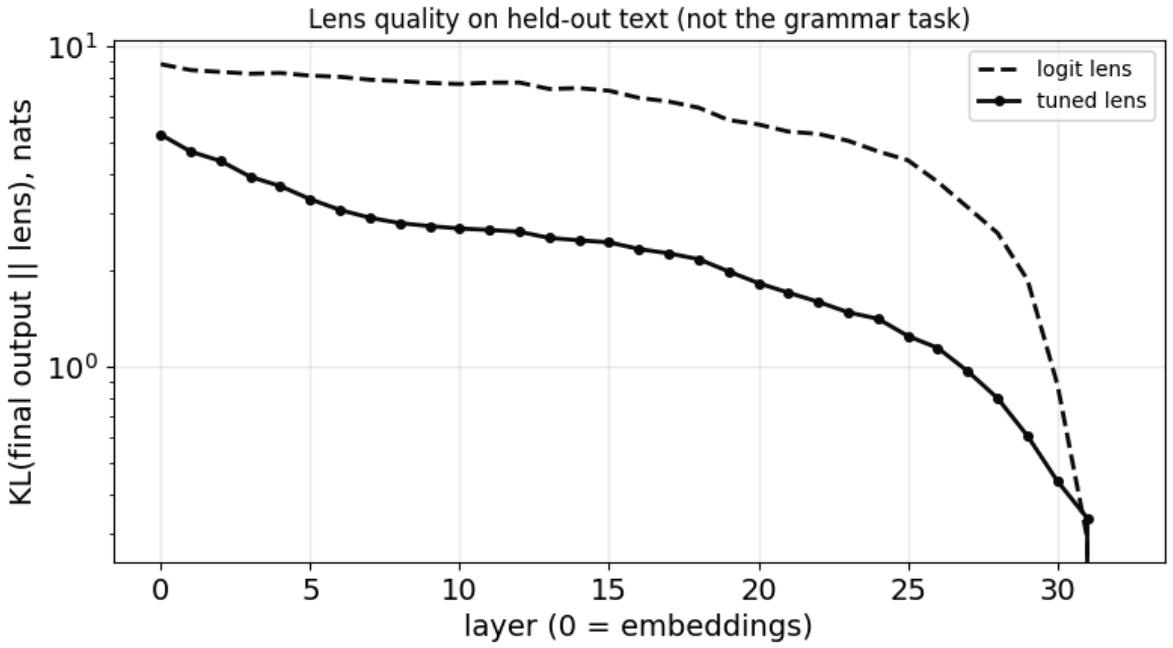
- **One-layer jump at 19** for both grammars.
- **Steady improvement from 19 to 32;** SRTT is higher at every layer, as expected for a fully predictable sequence.
- **Same onset and same shape for both grammars**: the simple repeated sequence is not resolved at an earlier layer than the probabilistic rules.
- **Controls (dashed) stay at ~0 at every layer**: the gain is specific to the grammar.
- Layers 0–5 sit slightly below 0 (~−0.06). This is not learning (no drift over time); word-pair frequencies simply differ a little between the random and grammar parts.

Part 3: Checking the result with a tuned lens

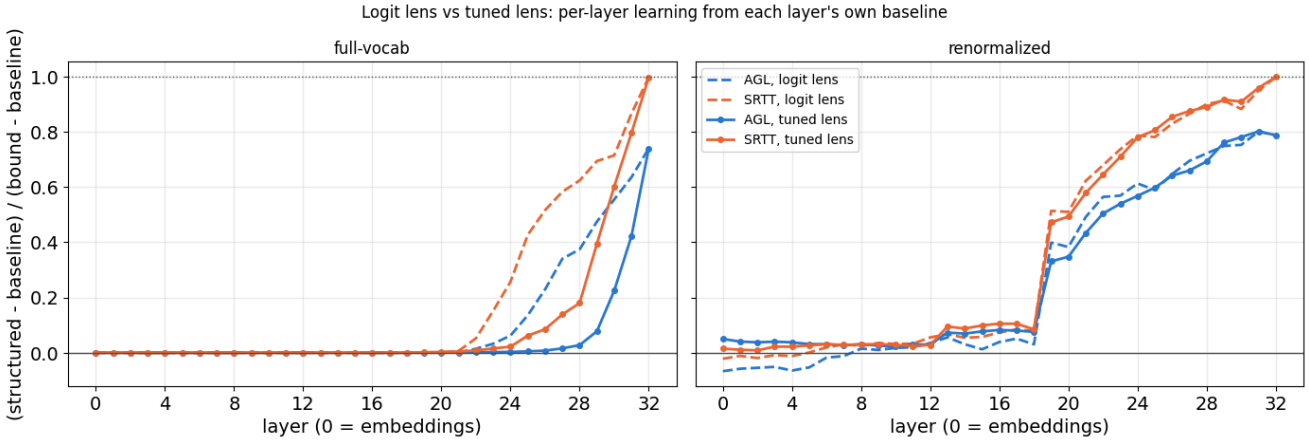
Why. The logit lens reads every layer with the last layer's translator. Inner layers may store information in a different format, so a layer can look empty when it isn't. Was the jump at layer 19 real, or an artifact of how we read the layers?

What a tuned lens is. A tuned lens (Belrose et al., 2023) learns, for each layer, a small linear correction (one matrix plus an offset) that converts that layer's state into the last layer's format before the same final step. It is trained to match the model's final output on

Wikipedia text (~400k tokens), never on the grammar, so it can only translate, not learn the grammar. Same forward passes, same two scores and same summary number as in Part 2.



Lens quality. On held-out Wikipedia text, the tuned lens is much closer to the model's real output than the logit lens at every layer up to 30 (e.g. layer 15: 2.4 vs 7.3 nats; lower is better). It reads inner layers far more faithfully.



Dashed lines = logit lens, solid lines = tuned lens.

- **Renormalized (right): the two lenses nearly overlap.** Same jump at layer 19, same rise to layer 32. A better translator does not reveal the grammar earlier, so the jump is a property of the model, not of the logit lens.
- **A small step at layer 13**, seen with both lenses and both grammars (to ~0.07–0.10 of the bound, flat until layer 18): a weak but consistent early trace of the structure.
- **Full-vocabulary (left): the tuned lens rises later** (10% of the bound at layer 30 for AGL and 27 for SRTT, versus 25 and 23 for the logit lens). This is expected: trained on English, its translation favours ordinary English words over copying the 4 random words from the context. For this task, the renormalized score is the relevant one.

Caveats. The lens was trained on ~400k tokens (the paper used ~65M), so it is not fully converged; at layer 31 it is slightly worse than the logit lens (0.34 vs 0.29 nats). Its quality

was measured on Wikipedia, not on our word streams. Both lenses are linear and aimed at next-word prediction, so structure stored in another form could still be missed.

Conclusions

1. **The paper's result reproduces:** a frozen Mistral-7B learns both grammars from context alone, learns which continuations are allowed, transfers the structure to new words, and penalizes rule violations.
2. **In-context grammar learning becomes readable abruptly at layer 19** (of 32), for both a deterministic sequence and probabilistic rules. Below it, layers are at or near chance. The same jump appears with a tuned lens (Part 3), so it is a property of the model, not of how we read it.
3. **It sharpens steadily from layer 19 to 32,** reaching the final performance only at the last layer.
4. **There is a gap between "knowing" and "saying":** the correct answer is preferred among the candidates from layer 19, but becomes the actual word output only from layer ~22.
5. **Both grammars share the same onset layer and depth profile,** consistent with one common mechanism rather than separate ones for simple and complex patterns.
6. A small, consistent step at layer 13 (to ~0.07–0.10 of the bound), seen with both lenses and both grammars: a weak early trace of the structure before the main jump.

Limitations and next steps

- **Readable is not computed.** The lens may miss structure held in a different form in layers 0–18. The tuned lens (Part 3) reads inner layers better and confirms the onset, but both lenses are linear and aimed at next-word prediction. Next: a linear probe, a small classifier trained per layer to predict the grammar state (q0–q4) directly from the layer's vector, trained and tested on different runs so it cannot memorize words. If it also jumps at layer 19, layer 19 is where the grammar is computed; if it succeeds earlier, the structure is computed earlier and layer 19 is where it is turned into a word prediction.
- **What happens at layer 19?** Next: inspect the attention heads in layers 17–19 (e.g. copying/induction-style heads), and repeat the per-layer analysis on another model (Llama-3.1-8B) to see whether the jump sits at the same relative depth.
- **Untrained-model control** for the per-layer analysis is not yet run.
- **Statistics:** the p-values on the vocabulary-switch and valid/swapped plots treat neighboring positions as independent, so they are far too small. To fix: one number per run, then a paired test across the 100 runs. The visible effects are large regardless.
- **SRTT runs share one sequence** (only the words differ), so SRTT error bars are narrower than for independent runs.
- **8-bit model** (memory limits) versus 16-bit in the paper; results match the paper closely, but this is untested.
- **No counting baseline** yet: a simple word-pair or word-triple counter would show how much of the AGL curve basic statistics alone can explain.